

Question 1: Compare and contrast NLTK and spaCy

Feature	NLTK (Natural Language Toolkit)	spaCy
Ease of Use	Steeper learning curve; more granular control	User-friendly and intuitive API
Features	Rich in linguistic resources (WordNet, corpora, POS taggers)	Industrial-strength NLP pipeline (tokenization, NER)
Performance	Slower; not optimized for production	Fast and efficient; built for real-world applications
Customization	Highly customizable; academic focus	Limited customization; optimized defaults
Use Case	Research, prototyping, teaching	Production-ready NLP systems
Aspect	NLTK (Natural Language Toolkit)	spaCy
Purpose	Designed for teaching, research, and prototyping in NLP.	Focused on production-level NLP applications.
Features	Tokenization, stemming, POS tagging, parsing, semantic reasoning, WordNet integration.	Tokenization, POS tagging, dependency parsing, named entity recognition
Ease of Use	Very detailed with academic flexibility, but requires more configuration.	Simplified high-speed API; easier for real-world projects.
Performance	Slower; implemented mostly in Python.	Very fast; written in optimized Cython.
Data Handling	Ideal for experimentation and linguistic education.	Ideal for industrial-scale NLP pipelines.
Use Case	Academic research, teaching, algorithm prototyping.	Chatbots, NLP APIs, real-time text analysis.

Question 2: What is TextBlob?

TextBlob is a Python library built on top of NLTK and Pattern. It simplifies common NLP tasks such as:

- **Sentiment Analysis:** Returns polarity (positive/negative) and subjectivity scores.
- **Translation:** Uses Google Translate API for multilingual support.
- **Part-of-Speech Tagging:** Identifies grammatical roles.
- **Noun Phrase Extraction:** Extracts key phrases from text.

Key Features:

Tokenization, POS tagging, noun phrase extraction.

Sentiment analysis (polarity and subjectivity).

Translation and language detection (via Google API).

Text classification.

Its simplicity makes it ideal for quick NLP prototyping and educational use.



Question 3: Role of Stanford NLP

Stanford NLP is a suite of NLP tools developed by Stanford University. It's widely used in:

- **Academia:** For research in linguistics, machine learning, and AI.
- **Industry:** In applications like chatbots, search engines, and document analysis.
- **Strengths:**
 - High accuracy in POS tagging, dependency parsing, and NER.
 - Multilingual support.
 - Integration with Java-based systems and REST APIs.

Stanford NLP (CoreNLP) is a powerful suite of NLP tools developed by Stanford University.

It provides state-of-the-art linguistic analysis using machine learning models trained on vast corpora.

Role in Academic Projects:

Used in linguistic and AI research.

Offers pre-trained models for parsing, NER, sentiment, and dependency parsing.

Supports multiple languages, aiding cross-linguistic research.

Role in Industry:

Integrated into chatbots, information extraction systems, and search engines.

APIs (Java, Python via Stanza) make it easy to embed in enterprise NLP pipelines.

Used by large organizations for advanced text understanding tasks.

Key Features:

Part-of-speech tagging

Named entity recognition (NER)

Coreference resolution

Sentiment analysis

Dependency parsing

Question 4: Architecture of RNN

Recurrent Neural Networks (RNNs) are designed for sequential data. Key components:

- **Input Layer:** Accepts time-series or text data.
- **Hidden Layer:** Maintains a memory of previous inputs via recurrent connections.
- **Output Layer:** Produces predictions or classifications.

Question 5: LSTM vs GRU

Feature	LSTM (Long Short-Term Memory)	GRU (Gated Recurrent Unit)
Complexity	More complex (3 gates: input, forget, output)	Simpler (2 gates: reset, update)
Training	Slower due to more parameters	Faster and more efficient
Performance	Better for long sequences	Comparable for shorter sequences
Use Case	When long-term dependencies matter	When speed and simplicity are preferred



Question 6: TextBlob Sentiment Analysis

```
from textblob import TextBlob
```

```
text = """I had a great experience using the new mobile banking app.  
The interface is intuitive, and customer support was quick to resolve my issue.  
However, the app did crash once during a transaction, which was frustrating"""
```

```
blob = TextBlob(text)  
print("Polarity:", blob.sentiment.polarity)  
print("Subjectivity:", blob.sentiment.subjectivity)
```

```
Polarity: 0.21742424242424244  
Subjectivity: 0.6511363636363636
```

✓ Question 7: NLTK Tokenization and Frequency Distribution

```
import nltk  
from nltk.tokenize import RegexpTokenizer  
from nltk.probability import FreqDist  
  
# Download the 'punkt' resource (though not strictly needed for RegexpTokenizer, good practice)  
try:  
    nltk.data.find('tokenizers/punkt')  
except LookupError:  
    nltk.download('punkt')  
  
text = """Natural Language Processing (NLP) is a fascinating field that combines linguistics,  
computer science, and artificial intelligence. It enables machines to understand,  
interpret, and generate human language. Applications of NLP include chatbots,  
sentiment analysis, and machine translation. As technology advances, the role of NLP  
in modern solutions is becoming increasingly critical."""  
  
# Use RegexpTokenizer as an alternative to word_tokenize  
tokenizer = RegexpTokenizer(r'\w+')  
tokens = tokenizer.tokenize(text)  
  
fdist = FreqDist(tokens)
```

```
print("Tokens:", tokens)
print("\nFrequency Distribution:")
for word, freq in fdist.items():
    print(f"{word}: {freq}")
```

Tokens: ['Natural', 'Language', 'Processing', 'NLP', 'is', 'a', 'fascinating', 'field', 'that', 'combines', 'linguistics', 'computer', 'science', 'and', 'artificial', 'intelligence', 'It', 'enables', 'machines', 'to', 'understand', 'interpret', 'generate', 'human', 'language', 'Applications', 'of', 'include', 'chatbots', 'sentiment', 'analysis']

Frequency Distribution:

Natural: 1
Language: 1
Processing: 1
NLP: 3
is: 2
a: 1
fascinating: 1
field: 1
that: 1
combines: 1
linguistics: 1
computer: 1
science: 1
and: 3
artificial: 1
intelligence: 1
It: 1
enables: 1
machines: 1
to: 1
understand: 1
interpret: 1
generate: 1
human: 1
language: 1
Applications: 1
of: 2
include: 1
chatbots: 1
sentiment: 1
analysis: 1

```
machine: 1
translation: 1
As: 1
technology: 1
advances: 1
the: 1
role: 1
in: 1
modern: 1
solutions: 1
becoming: 1
increasingly: 1
critical: 1
```



Question 8: LSTM Model in Keras

```
import tensorflow as tf
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense

texts = [
    "I love this project",
    "This is an amazing experience",
    "I hate waiting in line",
    "This is the worst service",
    "Absolutely fantastic!"
]
labels = [1, 1, 0, 0, 1]

# Tokenization
tokenizer = Tokenizer()
tokenizer.fit_on_texts(texts)
sequences = tokenizer.texts_to_sequences(texts)
```

```

padded = pad_sequences(sequences, padding='post')

# Get the vocabulary size
vocab_size = len(tokenizer.word_index) + 1

# Convert data to TensorFlow Dataset
dataset = tf.data.Dataset.from_tensor_slices((padded, labels))
dataset = dataset.batch(32) # Batch the data

# Model
model = Sequential()
model.add(Embedding(input_dim=vocab_size, output_dim=8, input_length=padded.shape[1]))
model.add(LSTM(16))
model.add(Dense(1, activation='sigmoid'))

model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
model.fit(dataset, epochs=10, verbose=1)

```

Epoch 1/10

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:97: UserWarning: Argument `input\_length` is deprecated
warnings.warn(

1/1  3s 3s/step - accuracy: 0.6000 - loss: 0.6946

Epoch 2/10

1/1  0s 171ms/step - accuracy: 0.4000 - loss: 0.6934

Epoch 3/10

1/1  0s 23ms/step - accuracy: 0.4000 - loss: 0.6922

Epoch 4/10

1/1  0s 20ms/step - accuracy: 0.6000 - loss: 0.6910

Epoch 5/10

1/1  0s 19ms/step - accuracy: 0.6000 - loss: 0.6898

Epoch 6/10

1/1  0s 19ms/step - accuracy: 0.6000 - loss: 0.6885

Epoch 7/10

1/1  0s 29ms/step - accuracy: 0.6000 - loss: 0.6873

Epoch 8/10

1/1  0s 48ms/step - accuracy: 0.6000 - loss: 0.6861

Epoch 9/10

1/1  0s 24ms/step - accuracy: 0.6000 - loss: 0.6848

Epoch 10/10

1/1 — 0s 53ms/step - accuracy: 0.6000 - loss: 0.6835
<keras.src.callbacks.history.History at 0x7f45d8a11880>

Question 9: spaCy NLP Pipeline

```
import spacy

nlp = spacy.load("en_core_web_sm")
text = """Homi Jehangir Bhaba was an Indian nuclear physicist who played a key role in the
development of India's atomic energy program. He was the founding director of the Tata
Institute of Fundamental Research (TIFR) and was instrumental in establishing the
Atomic Energy Commission of India."""

doc = nlp(text)

print("Tokens and Lemmas:")
for token in doc:
    print(f"{token.text} → {token.lemma_}")

print("\nNamed Entities:")
for ent in doc.ents:
    print(f"{ent.text} ({ent.label_})")
```

```
Tokens and Lemmas:
Homi → Homi
Jehangir → Jehangir
Bhaba → Bhaba
was → be
an → an
Indian → indian
nuclear → nuclear
physicist → physicist
who → who
played → play
a → a
key → key
```


role → role

in → in

the → the

→

development → development

of → of

India → India

's → 's

atomic → atomic

energy → energy

program → program

. → .

He → he

was → be

the → the

founding → found

director → director

of → of

the → the

Tata → Tata

→

Institute → Institute

of → of

Fundamental → Fundamental

Research → Research

(→ (

TIFR → TIFR

) →)

and → and

was → be

instrumental → instrumental

in → in

establishing → establish

the → the

→

```
Atomic → Atomic  
Energy → Energy  
Commission → Commission  
of → of
```



Question 10: Chatbot Architecture for Mental Health

Architecture:

- **Input Layer:** User text input.
- **Preprocessing:**
 - Tokenization and lemmatization using spaCy.
 - Sentiment and intent detection using LSTM/GRU.
- **Model:**
 - GRU-based sequence model for understanding context.
 - Trained on mental health conversations and FAQs.
- **Response Generation:**
 - Rule-based fallback for sensitive topics.
 - Retrieval-based or generative model for empathetic replies.

Ethical Considerations:

- **Privacy:** Encrypt user data and anonymize logs.
- **Bias Mitigation:** Train on diverse datasets.
- **Safety:** Escalate to human support for crisis detection.
- **Transparency:** Inform users that it's an AI assistant.

